Meridian: A Data Agent System for Temporal Financial Analysis

Zhi-Hao Tsai, Mikhail Berlay, Yu Sun

Abstract

Financial markets generate a large volume of heterogeneous data, including SEC filings, news, and market signals, making it difficult for both investors and Large Language Models (LLMs) to produce timely and coherent analysis. Traditional Retrieval-Augmented Generation (RAG) approaches are often limited to single-step retrieval and struggle with multi-source reasoning.

In this project, we build **Meridian**[1], a data agent-based financial analysis system that coordinates multiple specialized components following three architectural pillars: recursive retrieval for hierarchical document decomposition (The Librarian), dynamic graph-based relational reasoning (The Map), and adversarial multi-agent synthesis (The Debate). The current implementation uses a Retriever $\rightarrow$ Strategist pipeline where the Retriever gathers evidence from SEC filings, market data, and news via tool-augmented search, and the Strategist synthesizes it into actionable recommendations.

We evaluate our system on U.S. equity data covering 8 large-cap stocks during the March 24–31, 2026 evaluation window — a period of high geopolitical volatility driven by the US-Iran conflict. Using a 5-dimension LLM-as-judge rubric validated by human review, we measure improvement across two development phases. Phase 0 (LLM-only baseline) scored 1.0/5 across all dimensions; Phase 1 (Vanilla RAG with SEC filings and portfolio data) improved to 1.8/5, with actionability reaching 3.2/5. Human evaluation confirmed LLM judge alignment and uncovered three implementation bugs that guided subsequent development priorities.

1 Introduction

The rapid growth of financial data, including regulatory filings, earnings transcripts, and market news, has made it increasingly difficult for both retail investors and Large Language Models (LLMs) to produce timely and coherent analysis. A single asset may generate thousands of data points daily, leading to information overload and delayed decision-making. Traditional approaches, including static LLM pipelines and standard Retrieval-Augmented Generation (RAG) systems [3], often struggle to integrate multi-source information and perform multi-step reasoning.

To address these challenges, we build **Meridian**, a data agent-based system for financial analysis across multiple data sources. Our goal is to provide a practical and interpretable system, and to evaluate whether a modular data agent design can improve performance over LLM-only baselines. The system follows a sequential pipeline architecture — Retriever, then Strategist, then Critic — where the Retriever gathers evidence using tool-augmented search, the Strategist synthesizes it into recommendations, and the Critic adversarially validates the output before a final revision pass. This design maps directly to the three pillars proposed in our project proposal: recursive retrieval (The Librarian), graph-based relational reasoning (The Map), and adversarial synthesis (The Debate).

Since Milestone 1, we have made three primary contributions: (1) the design and implementation of a reproducible evaluation framework with 5 scored dimensions, ground truth answers, and both

[1] https://cs6960.github.io/utah-cs6969-proj/

LLM and human judges; (2) the development and evaluation of two system phases — an LLM-only baseline and a Vanilla RAG pipeline with tree-structured retrieval over SEC filings and portfolio data; and (3) the incorporation of human feedback that uncovered implementation limitations invisible to automated scoring, directly informing our development roadmap.

2 Related Work

Recent advancements in large language models (LLMs) have enabled their application in financial analysis. FinGPT [4] demonstrates that domain-specific fine-tuning can improve financial understanding, but such models remain limited by static knowledge and lack temporal adaptability. Memory-augmented approaches such as FinMem [7] introduce layered memory structures to address long-term reasoning, but primarily focus on intra-task consistency and struggle to capture cross-entity and cross-sector relationships in dynamic markets.

Multi-agent frameworks, such as TradingAgents [6], explore the use of role-based agent interactions for financial decision-making. While these approaches improve reasoning diversity, they introduce additional complexity and are difficult to evaluate systematically. More generally, recent practical frameworks and educational resources have highlighted the emergence of data agent systems that combine planning, tool use, and modular components for data-centric tasks [1].

In parallel, recent work on retrieval and reasoning has explored more advanced techniques. Recursive language models [8] enable hierarchical decomposition for long-context understanding, while graph-based methods such as LazyGraphRAG [2] aim to capture relational structure across documents. However, these approaches are often computationally intensive or require complex preprocessing.

For evaluation, the LLM-as-judge paradigm [9] has emerged as a scalable alternative to human annotation. By prompting a language model with a rubric and reference answer, automated judges can score system outputs on multiple dimensions. We adopt this approach as our primary scoring method while retaining human review for validation and bug discovery.

Finally, practical perspectives on financial analysis emphasize the importance of integrating diverse data sources and computational tools for decision-making [5]. Despite these advances, existing systems still lack a unified and practical framework that integrates temporal awareness, multi-source retrieval, and controllable reasoning for financial analysis.

3 Methodology

Our system aims to address two key limitations in conventional LLM pipelines: (1) the inability to distinguish between outdated and recent financial information, and (2) limited capability to integrate and reason over multiple data sources.

We implement **Meridian** with Qwen 3.5-122B as a multi-agent pipeline architecture that coordinates specialized agents for evidence gathering, synthesis, and adversarial review. The design follows the three-pillar structure from our project proposal, with each pillar corresponding to a development phase.

3.1 System Overview

The system uses a sequential pipeline: given a user query, the Retriever agent gathers evidence from multiple data sources using tools, then the Strategist agent receives the structured evidence package and synthesizes it into an actionable recommendation. Only the Retriever has tools; the Strategist and (future) Critic are single-pass LLM calls receiving pre-gathered evidence as text, keeping costs predictable and reasoning traces auditable.

3.2 Pipeline Architecture and Proposal Mapping

The pipeline maps directly to the three pillars from our project proposal:

1. **The Librarian (Phases 1–2):** The Retriever agent implements recursive retrieval over SEC filings via a tree-structured RAG pipeline, and (in Phase 2) temporal news retrieval from a

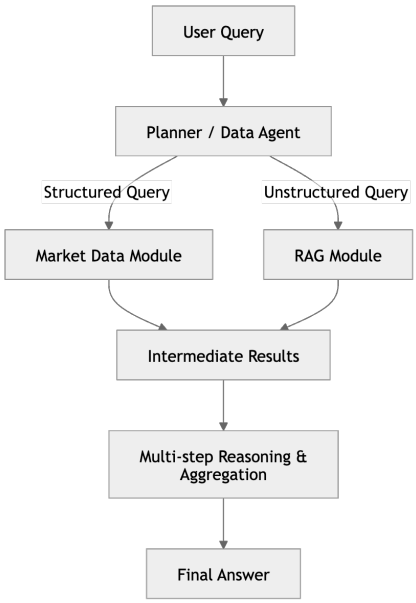

Figure 1: Overview of the Meridian system architecture. The Retriever gathers evidence via tool-augmented search; the Strategist synthesizes recommendations; the Critic (Phase 4) adversarially validates the output.

stored corpus. This addresses the proposal's goal of hierarchical document decomposition with temporal awareness.

2. **The Map (Phase 3):** An entity-relationship graph constructed from the news corpus enables the Retriever to traverse cross-sector connections at query time, implementing the proposal's LazyGraphRAG-inspired relational reasoning.
3. **The Debate (Phase 4):** A Critic agent adversarially challenges the Strategist's recommendation, followed by a revision pass. The final output includes both the revised recommendation and a dissenting perspective, implementing the proposal's adversarial multi-agent synthesis.

The pipeline currently implements Phases 0–1 (Retriever → Strategist). Phases 2–4 will incrementally add tools and agents to this pipeline without changing its fundamental sequential structure.

3.3 Tree-Structured RAG Pipeline

The retrieval module uses a hierarchical indexing strategy for SEC filings. Each 10-K report is processed into a three-level tree:

1. **Document node:** One root node per filing containing a compact outline.
2. **Section nodes:** One node per filing section (e.g., Risk Factors, MD&A) with full text and a summary embedding.
3. **Chunk nodes:** Fine-grained text chunks under each section, optimized for vector retrieval.

Retrieval works bottom-up: the user query is embedded and matched against chunk nodes via cosine similarity using the `match_document_tree_nodes` Supabase RPC. Parent nodes are then fetched to reconstruct the document → section → chunk lineage, enabling the agent to cite both the specific excerpt and its broader filing context.

3.4 Market Data Module

The market data module provides structured financial information including current stock prices, daily price changes, and portfolio holdings with cost basis. Historical daily closing prices for the

evaluation window (March 24–31, 2026) are stored in a Supabase table, enabling the agent to cite specific price movements over the evaluation period.

3.5 Development Progression

Phase 0 used a single monolithic advisor agent that had all tools available but failed to invoke them, serving as the LLM-only baseline. Phase 1 introduced the Retriever → Strategist pipeline with SEC filing retrieval tools, corresponding to the Vanilla RAG baseline from our proposal. The Retriever systematically gathers portfolio holdings, stock prices, and SEC filing excerpts, then the Strategist synthesizes this evidence into recommendations. A deterministic fallback ensures tool calls are made even when the Retriever agent fails to invoke them autonomously, guaranteeing the Strategist always receives an evidence package.

4 Experimental Design

4.1 Dataset

We evaluate Meridian on a static U.S. equity dataset covering 8 large-cap stocks: AAPL, MSFT, NVDA, AMZN, GOOGL, JPM, LLY, and XOM. Our proposal originally scoped the dataset to 5–10 stocks over Q4 2025. We refined this to 8 stocks over a one-week window (March 24–31, 2026) for two reasons: (1) the project runs on Supabase Free (Nano) tier with a 3-second statement timeout, requiring a compact corpus that stays within storage and query limits; and (2) the 8 portfolio tickers span four sectors (technology, energy, healthcare, financials), providing sufficient cross-sector diversity to capture news-driven causal dynamics within a single week. The evaluation window was selected for its high news density across all tickers and clear cross-sector dynamics driven by the US-Iran conflict.

For each company, we collect three types of data:

1. **SEC filings:** 10-K annual reports embedded in the tree-RAG pipeline. All 8 holdings have indexed filings in the `document_tree_nodes` Supabase table.
2. **News articles:** 80 relevant articles (10 per ticker) plus 10 noise articles stored in the `news_articles` table. Noise includes off-topic articles (NASA, FIFA, agriculture) and not-in-portfolio ticker articles (TSLA, PFE).
3. **Market data:** Daily closing prices for all 8 tickers across the 7-trading-day evaluation window, plus current price snapshots with day change data.

Using a fixed time window and stored corpus ensures reproducibility across evaluation runs and prevents temporal leakage.

4.2 Evaluation Framework

We designed a staged evaluation framework where the same set of 4 preset portfolio-level questions is run against the system after each development phase. The preset questions are:

1. "What is my biggest portfolio risk?"
2. "Am I diversified enough?"
3. "Which holdings look strongest?"
4. "Where should new cash go?"

For each question, we authored ground truth reference answers grounded in the March 24–31 evaluation window. The ground truth captures the dominant market theme — the US-Iran war, including the Strait of Hormuz closure, oil price surge, IRGC retaliatory threats to tech companies, and flight-to-quality dynamics into defensive holdings (JPM, LLY, XOM).

4.3 Scoring Rubric

Each response is scored 1–5 on five dimensions:

1. **Groundedness:** Does the response cite real, specific data (prices, events, percentages) rather than generic advice?
2. **Completeness:** Does it cover the key factors from the ground truth reference answer?
3. **Actionability:** Does it give specific, usable recommendations (e.g., "trim NVDA", "add to JPM") rather than vague platitudes?
4. **Temporal Precision:** Does the response cite facts from the correct time window (March 24–31, 2026) with specific dates?
5. **Relational Recall:** Does the response identify cross-sector causal chains (e.g., Iran conflict → oil surge → XOM benefit + tech selloff)?

The first three dimensions were defined at Milestone 1. Temporal Precision and Relational Recall were added for Milestone 2 to measure the system's ability to reason temporally and relationally — capabilities that standard RAG metrics do not capture.

4.4 Scoring Method

Primary: LLM-as-Judge. Automated scoring using Qwen 3.5-122B via NVIDIA API. The judge receives the preset question, ground truth answer, date-stamped temporal facts the agent should cite, cross-sector relational connections the agent should identify, and the agent's response. It returns integer scores for all 5 dimensions plus a brief explanation.

Secondary: Human Review. Manual evaluation by a team member with finance domain knowledge (finance minor, market experience). Human review serves two purposes: (1) validating LLM judge score alignment, and (2) uncovering implementation-level issues that automated scoring cannot detect, such as tool failures, empty retrieval results, or data pipeline bugs.

4.5 Additional Diagnostics

Beyond the 5 scored dimensions, the evaluation tracks two diagnostic metrics:

1. **Noise Citation Detection:** The eval runner scans agent responses for mentions of non-portfolio tickers from the noise corpus (TSLA, PFE) and other well-known tickers not in the portfolio. A well-performing agent should produce zero noise citations.
2. **Tool Usage Tracking:** The backend returns the list of tools invoked during each query. This distinguishes "agent did not use tools" (routing problem) from "agent used tools but answer quality was low" (tool quality problem).

4.6 Development Stages

The system is evaluated after each development phase. Stages are sequential — a stage cannot begin until the previous stage passes its gate criteria.

The development stages map to the baselines defined in our project proposal:

Table 1: Development stages mapped to proposal baselines and architectural pillars.

Phase	Stage	Proposal Baseline	Proposal Pillar	Primary Dimension
0	`baseline`	LLM-only (added)	—	— (floor)
1	`rag_reports`	Vanilla RAG	The Librarian	Groundedness
2	`news_agent`	Temporal RAG	The Librarian	Temporal Precision
3	`graph`	—	The Map	Relational Recall
4	`critic`	Full System	The Debate	Actionability (all)

Phase 0 was added beyond the original proposal to establish an LLM-only floor. Phases 1–2 correspond to the proposal's Vanilla RAG and Temporal RAG baselines, respectively. Phases 3–4 together deliver the Full System with graph reasoning and adversarial synthesis.

5 Results

5.1 Phase 0: Baseline

The baseline evaluation used a single advisor agent with DuckDuckGo search, Yahoo Finance news, and stock price tools. The agent scored **1.0 across all 5 dimensions** from both the LLM judge and human reviewer.

The agent failed to use any of its available tools across all 4 questions. Instead of querying stock prices, SEC filings, or news, it asked the user for portfolio data it already had access to. All responses were generic templates with zero grounding in the March 24–31 evaluation window. One noise citation (TSLA) was detected.

Human evaluation (Mikhail Berlay — finance minor, market experience) confirmed the LLM judge scores, establishing a validated floor for subsequent comparison.

5.2 Phase 1: RAG Reports

Phase 1 replaced the monolithic advisor with the Retriever → Strategist pipeline incorporating tree-structured RAG retrieval for SEC filings. This corresponds to the Vanilla RAG baseline from our proposal: the Retriever gathers portfolio holdings, stock prices, and SEC filing excerpts; the Strategist synthesizes this evidence into a recommendation.

Table 2: Evaluation results across development phases. Scores are averages across 4 preset questions (1–5 scale).

Stage	Ground.	Compl.	Action.	Temp.P	Rel.R	Avg	Noise	Tools
`baseline`	1.0	1.0	1.0	1.0	1.0	1.0	1	0/4
`rag_reports`	2.0	1.5	3.2	1.0	1.0	1.8	1	4/4

Key observations from Phase 1:

1. **Tool usage fixed:** Tools were called in all 4 questions (up from 0/4), confirming that the Retriever → Strategist pipeline resolved the baseline's tool invocation failure.
2. **Actionability jumped to 3.2:** The agent now gives concrete directional recommendations grounded in portfolio data and SEC filing excerpts, rather than generic advice.
3. **Temporal Precision and Relational Recall remain at 1.0:** This is expected — these dimensions require news corpus integration (Phase 2) and entity-relationship graph data (Phase 3), respectively. Neither data source is connected yet.
4. **One noise citation (PFE):** In question 4 ("Where should new cash go?"), the agent cited Pfizer, which is not a portfolio holding. This indicates the agent occasionally references out-of-portfolio companies when generating recommendations.

5.3 Human Evaluation and Bug Discovery

Human evaluation at Phase 1 scored the system at an average of **1.9/5** across questions (Q1: 2.5, Q2: 1.0, Q3: 2.0, Q4: 2.0), closely aligning with the LLM judge average of 1.8/5. This alignment validates the LLM-as-judge approach for our domain.

Critically, human evaluation uncovered three implementation bugs that the LLM judge could not detect:

1. **Cash excluded from portfolio math:** The $12,500 cash position was not counted in the portfolio total, causing all percentage calculations (concentration, allocation) to be systematically wrong.
2. **10-K filing excerpts empty:** The RAG retrieval tool returned report titles but not actual filing content. The agent received metadata about available reports but no substantive text to reason over.

3. **Historical prices not surfacing:** Seven days of daily closing prices were available in the database but the agent only accessed single-point snapshots, preventing it from citing price trends or weekly movements.

These bugs explain the gap between the theoretical capability of the pipeline (SEC filings indexed, historical prices stored) and the actual eval scores. The LLM judge scored responses as poor because the content was weak, but only human review identified *why* the content was weak — the data existed but was not reaching the agent.

5.4 Incorporating Human Feedback

The three bugs identified through human evaluation became the team's primary development priority. Rather than advancing to Phase 2 (news integration), the team decided to fix these root causes first, since they affect the foundation that all subsequent phases build upon.

This decision validated a key design choice in our evaluation framework: maintaining human review as a secondary scoring method alongside automated LLM judging. The LLM judge is effective for measuring response quality at scale, but human domain experts catch implementation-level issues that manifest as quality problems but have infrastructure root causes.

6 News Data Structure

In preparation for Phase 2 (news agent), we have implemented the news corpus infrastructure, though it is not yet connected to the agent as a tool.

6.1 Schema and Seeding

The `news_articles` table in Supabase stores articles with fields for ticker, headline, body text, source, publication timestamp, relevance flag, and metadata tags. Three indexes support efficient querying: by ticker, by publication date, and by relevance.

The seeding pipeline (`script/seed_news.py`) fetches articles from Yahoo Finance for each portfolio ticker and inserts them with `relevant = true`. A separate `-noise` mode inserts predefined noise articles with `relevant = false`.

6.2 Current Corpus

The corpus contains 90 articles: 80 relevant (10 per portfolio ticker) covering the March 24–31 evaluation window, and 10 noise articles (6 off-topic with ticker `NONE`, 2 TSLA, 2 PFE). Relevant articles cover the Iran war's impact on each holding — IRGC threats to tech companies, oil price surge, flight-to-quality dynamics.

6.3 Gaps for Phase 2 Completion

The news corpus data is stored and queryable, but the following components are needed to complete Phase 2:

1. **News query tool:** A `query_news_articles` LangChain tool that queries the Supabase table by ticker and date range. This tool must *not* filter by `relevant = true` — the agent should demonstrate its ability to ignore noise.
2. **Tool registration:** Adding the news tool to the agent's available tool set.
3. **Agent prompt update:** Instructing the agent to query news articles when answering portfolio questions requiring temporal context.

7 Future Work

Our remaining development is organized into three phases, each building incrementally on the previous.

7.1 Phase 2: News Agent (Immediate Priority)

Before beginning Phase 2, the three bugs identified during human evaluation must be resolved: correcting portfolio cash calculations, ensuring RAG retrieval returns actual filing content, and exposing historical price data to the agent. These fixes are prerequisites because they affect the data foundation that the news tool will build upon.

Once the fixes are verified, Phase 2 adds the `query_news_articles` tool to the Retriever, enabling temporal awareness. In addition, we will implement recency-weighted re-ranking of retrieved chunks, as described in our proposal's recursive retrieval design (Section 4.1): after initial cosine similarity matching, results are re-ranked using a recency score derived from the publication timestamp, promoting recently published and contextually relevant chunks over stale historical data. The primary eval target is Temporal Precision > 2.5, with responses citing specific dates and events from the March 24–31 window. We also target zero noise citations, demonstrating the agent's ability to filter irrelevant articles from the mixed corpus.

7.2 Phase 3: Entity-Relationship Graph

Phase 3 builds a static entity-relationship graph from the news corpus at ingestion time, following the LazyGraphRAG-inspired approach from our proposal. Graph construction uses named entity recognition and noun phrase co-occurrence to identify financial entities (companies, sectors, events, products) and infer relationships (`supplier`, `competitor`, `affects`, `benefits_from`, `threatened_by`), supplemented by LLM-based triple extraction for complex relationships that NER alone cannot capture. A 1–2 hop graph traversal tool enables the Retriever to discover cross-sector causal chains at query time without requiring full community detection. The primary eval target is Relational Recall > 3.0, with responses explicitly tracing multi-hop cause-effect relationships (e.g., Iran conflict $\rightarrow$ Hormuz closure $\rightarrow$ oil surge $\rightarrow$ XOM benefit + tech selloff).

7.3 Phase 4: Adversarial Critic

Phase 4 wires the critic agent into the full pipeline: Retriever $\rightarrow$ Strategist $\rightarrow$ Critic $\rightarrow$ Revision. After the strategist produces a recommendation, the critic adversarially challenges it, then the strategist revises incorporating valid criticisms. The final output includes both the revised recommendation and a dissenting perspective. The target is monotonic improvement across all 5 dimensions relative to Phase 3, with Actionability > 3.5.

8 Conclusion

In this milestone, we presented the evaluation framework and initial development results for **Meridian**, a multi-agent system for temporal financial analysis. Our primary contributions are: (1) a reproducible 5-dimension evaluation framework with LLM-as-judge scoring validated by human review; (2) quantified improvement from Phase 0 (LLM-only baseline, 1.0/5 avg) to Phase 1 (Vanilla RAG, 1.8/5 avg), with tool usage going from 0/4 to 4/4 and actionability reaching 3.2/5; and (3) the demonstration that human evaluation uncovers implementation-level bugs invisible to automated scoring, directly informing development priorities.

The system implements a Retriever $\rightarrow$ Strategist pipeline that maps to the first pillar of our proposal (The Librarian). The news corpus infrastructure is implemented and ready for Phase 2 integration, which will add temporal awareness through news retrieval and recency-weighted re-ranking (completing The Librarian). Subsequent phases will add graph-based cross-sector reasoning (Phase 3, The Map) and adversarial critique with a Critic agent and revision pass (Phase 4, The Debate), delivering the full system proposed in our project proposal.

References

[1] DeepLearning.AI. Building and evaluating data agents. https://www.deeplearning.ai/short-courses/, 2024.

[2] Darren Edge, Ha Trinh, and Jonathan Larson. Lazygraphrag: Setting a new standard for quality and cost. https://www.microsoft.com/en-us/research/blog/lazygraphrag-setting-a-new-standard-for-quality-and-cost/, 2024.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, and Lewis. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[4] Xiao-Yang Liu, Guoxuan Wang, and Daochen Zha. Fingpt: Democratizing internet-scale data for financial large language models. *arXiv preprint arXiv:2307.10485*, 2023.

[5] Stefan Papp. *Investing for Programmers: Understanding Markets Through Data and Code*. Self-published, 2025.

[6] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.

[7] Yangyang Yu, Haohang Li, et al. Finmem: A performance-enhanced llm trading agent with layered memory and character design. *arXiv preprint arXiv:2311.13743*, 2023.

[8] Alex L. Zhang et al. Recursive language models: Unlocking infinite context through programmatic decomposition. *arXiv preprint*, 2025.

[9] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, and Dacheng Li. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.